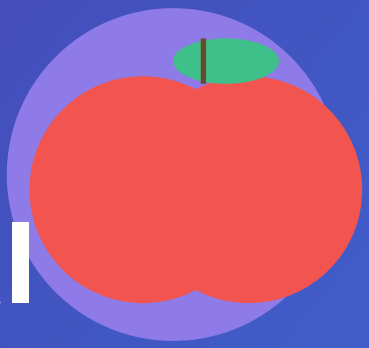
COMPUTER VISION · OBJECT DETECTION



FruitVision AI

Fruit Detection and Counting System

Prerequisites + Complete Project Viva & Interview Preparation

145 questions with ready-to-speak answers, in 12 parts.

YOLO26n | Streamlit | OpenCV | Apple | Banana | Orange

SETUP

Prerequisites — Fruit Detection and Counting System

1. Software requirements

- Python 3.10 or compatible version
- Streamlit for the user interface
- Ultralytics YOLO for object detection
- OpenCV for video processing
- NumPy and Pillow for image processing
- Pretrained YOLO model weights

2. Hardware requirements

Component	Recommended requirement
Operating system	Windows 10/11, Linux or macOS
Processor	Intel Core i3 or equivalent, preferably i5
RAM	8 GB recommended; 4 GB may work for basic testing
Storage	2–5 GB free space recommended
GPU	Not mandatory; NVIDIA GPU can improve processing speed
Camera	Optional, for camera-based input

OVERVIEW

Project at a glance

Know these facts before answering any question.

PROJECT TYPE **Object detection and counting**	MODEL **YOLO26n pretrained**	USER INTERFACE **Streamlit**
IMAGE/VIDEO PROCESSING **OpenCV, Pillow, NumPy**	CONFIGURED CLASSES **Apple, banana, orange**	DEFAULT THRESHOLD **0.25 (25%)**

IMPORTANT

Important: this version uses a pretrained model and filters its detections to three fruit classes. It does not currently load a custom-trained model based on the 30-category fruit and vegetable dataset. Also, video detection processes frames individually; it does not track each unique fruit across the entire video.

Contents

Part 1	Basic and non-technical questions	Q1–Q10
Part 2	Why YOLO? The most important questions	Q11–Q24
Part 3	Why only apples, bananas and oranges?	Q25–Q32
Part 4	Dataset and computer vision fundamentals	Q33–Q44
Part 5	Confidence threshold, accuracy and evaluation	Q45–Q57
Part 6	Counting logic and difficult edge cases	Q58–Q69
Part 7	Streamlit and application architecture	Q70–Q90
Part 8	Code-level questions an expert may ask	Q91–Q100
Part 9	Technology stack and environment	Q101–Q110
Part 10	Non-technical presentation and business questions	Q111–Q120
Part 11	Advanced questions from a trained AI/ML expert	Q121–Q135
Part 12	Challenging questions that test honesty	Q136–Q145

PART 1 Basic and non-technical questions

Q1 What is your project?

Answer:

SAY THIS

"My project is FruitVision AI, a computer vision-based fruit detection and counting system. It uses a pretrained YOLO object detection model to identify apples, bananas and oranges in images and videos. The application draws bounding boxes around detected fruits, counts them class-wise and provides a user interface to visualize and download the results."

Example: If an image contains three apples, two bananas and one orange, the system may report six detected fruits in total, provided all six are detected correctly.

Q2 What problem does your project solve?

Manual fruit counting can be time-consuming, especially when many fruits appear in an image or video.

My project automates part of that process by detecting visible fruits and generating counts. It can be useful as a starting point for fruit inventory, produce inspection, educational computer vision demonstrations and agricultural automation prototypes.

Q3 Why did you choose this project?

SAY THIS

"I chose this project because it combines computer vision, deep learning and application development. I wanted to understand how an AI model can identify objects in visual data and how its predictions can be converted into a practical application."

Q4 What is the main objective?

The main objective is to:

- Detect selected fruit classes.
- Draw bounding boxes around them.
- Count detections by fruit type.
- Process both images and videos.
- Provide a simple interface for uploading inputs and viewing results.

Q5 Who can use this project?

Potential users include fruit retailers, warehouse operators, agricultural technology developers, students and researchers.

However, the current version is a prototype. Before using it for commercial inventory, we would need to test it on real-world fruit images and measure its accuracy under different conditions.

Q6 Is this a complete commercial product?

SAY THIS

“No. It is a functional prototype demonstrating fruit detection and counting. A commercial version would require more extensive evaluation, broader fruit-class support, reliability testing, and possibly tracking, database integration and deployment infrastructure.”

NOTE

This is a stronger answer than claiming the prototype is production-ready without evidence.

Q7 What is the input and output?

Stage	Input/output
Input	Uploaded image, video or camera snapshot
Processing	YOLO inference and class filtering
Visual output	Image or video with bounding boxes
Numerical output	Total detections and fruit-wise counts
Export	Annotated image or processed video download

Q8 What is the uniqueness of your project?

SAY THIS

“The value is not just the detection model. It is the integration of object detection, fruit-wise counting, image/video processing and an interactive interface into one workflow.”

NOTE

Be honest: YOLO is an existing model, not an algorithm invented for this project.

Q9 What did you learn from this project?

SAY THIS

“I learned how pretrained object detection models are integrated into applications, how confidence thresholds affect predictions, how images and video frames are processed, and how to build an interface that makes model output understandable to users.”

Q10 What are the limitations?

The current version:

- Supports only three configured fruit classes.
- Can miss fruits because of occlusion, lighting, unusual angles or small object size.
- Can produce false detections.
- Does not uniquely track fruit identities across video frames.
- Has not been demonstrated to meet a measured accuracy target on a dedicated fruit test dataset.
- Depends on the capabilities of its pretrained model.

PART 2 Why YOLO? The most important questions

Q11 What is YOLO?

YOLO stands for You Only Look Once. It is a family of deep-learning object detection models.

An object detector predicts what objects are present in an image and where they are located. YOLO produces predictions that include bounding boxes, class labels and confidence scores.

Example:



Conceptual example of object detection; this illustration is not an output captured from your running application.

Q12 Why is it called “You Only Look Once”?

The name comes from the YOLO approach of predicting object locations and classes in a single network inference stage, rather than using a traditional proposal-first pipeline.

Modern YOLO implementations are more sophisticated than the name alone suggests, but the key idea is efficient object detection.

Q13 Why did you use YOLO?

This is one of the most important questions for your project.

SAY THIS

“I used YOLO because the project requirement was to use a YOLO-based detector. YOLO is suitable for object detection because it predicts object locations and classes efficiently, and it has pretrained models that allow us to build a working prototype without training a model from scratch.”

SAY THIS

“For this version, I used a pretrained model so I could focus on the complete detection workflow and application interface. Training a fruit-specific model for many categories would require collecting or selecting a suitable dataset, preparing annotations, training and evaluating the model, and more development time.”

NOTE

Do not say you selected YOLO after a benchmark proved it was the best model unless you actually performed that comparison.

Q14 Why not use CNN classification instead?

A traditional image classifier generally predicts the main class of an image, such as “apple.”

But our image might contain five apples, two bananas and an orange. We need the model to locate each individual object and identify its class.

Object detection is therefore more suitable than simple image classification for this counting task.

Q15 What is the difference between classification and detection?

Image classification	Object detection
Predicts an image or crop's class	Predicts classes and locations
May output “apple”	Can locate several apples and bananas
Does not inherently count individual objects	Supports counting detected instances

Q16 What is the difference between object detection and segmentation?

Object detection gives a bounding rectangle around an object.

Image segmentation identifies the object's pixels or boundary more precisely.

For basic fruit counting, bounding boxes are a reasonable starting point. If we need exact fruit surface area or fruit boundaries when objects overlap, instance segmentation could be useful.

Q17 Why YOLO instead of Faster R-CNN?

Faster R-CNN is a two-stage detector, while standard YOLO models use a more direct detection pipeline. YOLO is commonly chosen when a balance of speed and detection quality is important.

However, neither model is always better. A fair comparison would test both on the same dataset, hardware and evaluation conditions.

Q18 Why YOLO instead of SSD?

SSD is another single-stage detector. Both approaches aim for efficient detection, but their performance depends on the model version, input resolution, hardware and dataset.

I would compare precision, recall, mAP, inference time and resource usage before claiming one is superior for this project.

Q19 What is YOLO26n?

yolo26n.pt is the pretrained model file used by the code. The n suffix denotes the nano-sized model variant.

Nano variants are designed to be relatively lightweight. They are useful for prototypes and constrained hardware, although the actual speed and accuracy depend on the environment and input data.

Q20 What does .pt mean?

.pt is a file extension commonly used for PyTorch model checkpoints. In this project, Ultralytics loads the pretrained model from yolo26n.pt.

The file contains learned model parameters and associated checkpoint information, rather than a dataset of fruit images.

Q21 Did you train YOLO yourself?

Correct answer for the current implementation:

SAY THIS

“No. The current application uses a pretrained YOLO26n model. I integrated it into the detection pipeline and configured the application to return only the three selected fruit classes. I have not integrated custom-trained fruit model weights into this version.”

NOTE

Never claim that you trained the model unless you actually trained it and can show the training configuration and results.

Q22 What is transfer learning?

Transfer learning uses knowledge learned by a model on one dataset as a starting point for another task.

For example, we could fine-tune a pretrained YOLO model using annotated images containing additional fruits. The current application uses pretrained weights directly; it does not perform fine-tuning at runtime.

Q23 What is the difference between pretrained and custom-trained models?

- Pretrained model: already has learned weights from prior training.
- Fine-tuned model: starts from existing weights and is further trained on task-specific data.
- Model trained from scratch: learns its weights through training without starting from a pretrained checkpoint.

Our current implementation uses the first option.

Q24 How does YOLO detect fruits internally?

At a high level:

1. The image is passed to the model.
2. The model processes visual patterns through its neural network.
3. It predicts candidate object boxes, class scores and confidence values.
4. Post-processing filters detections based on the configured confidence threshold and other settings.
5. The application draws boxes and counts the remaining detections.

The exact internal layers and prediction pipeline depend on the YOLO model implementation.

PART 3 Why only apples, bananas and oranges?

Q25 Why is your project detecting only these three fruits?

Your prepared answer:

> **SAY THIS**
> *"The requirement was to use YOLO, so I chose a pretrained YOLO model to implement the detection system. A pretrained model is a practical starting point, but extending the system to many fruit categories requires more work: selecting a suitable dataset, checking its labels, preparing the training and validation data, fine-tuning the model, evaluating it and integrating the resulting weights."*

SAY THIS

“For this version, I limited the application to apples, bananas and oranges so that I could build and demonstrate the complete image, video and user-interface workflow within the available time. The architecture can be extended to additional fruit classes in a future version.”

NOTE

This is the answer you wanted, with an important technical detail: the current code filters detections to these three classes; it does not use a custom-trained 30-category fruit model.

Q26 Does YOLO detect only three fruits by default?

No. YOLO models can support different classes depending on the weights and training dataset.

In this application, the three-class restriction comes from this code:

```
FRUIT_CLASSES = [46, 47, 49]
```

Those are the relevant class IDs in the pretrained model's COCO class mapping: banana, apple and orange.

The model itself is not inherently restricted to three fruits.

Q27 Why did you not use the 30-category fruit and vegetable dataset?

SAY THIS

“The project scope focused on implementing a YOLO-based working system. Training and validating a model across 30 categories would require preparing the dataset, confirming its annotation format, configuring the training pipeline, selecting hyperparameters and evaluating performance. I kept this version focused on the end-to-end application and three supported classes.”

NOTE

If asked whether that dataset was used to train the current model, say no, not in this version.

Q28 Is collecting more data enough to support more fruits?

Not by itself. The data needs appropriate annotations, and the model must be trained or fine-tuned to recognize the new classes. The inference code must also use the correct class names and weights.

Q29 How would you add mangoes, grapes or strawberries?

I would:

1. Obtain or create a properly licensed dataset containing those classes.

2. Check the images and annotations.
3. Convert annotations to the format required by the selected YOLO training pipeline.
4. Split data into training, validation and test sets.
5. Fine-tune the pretrained model.
6. Evaluate precision, recall and mAP.
7. Integrate the trained weights into the application.
8. Update class filters and the results UI.
9. Test it on unseen fruit images.

Q30 Can you simply add more class IDs to the list?

Only if the selected model's class mapping actually contains those classes. Adding an arbitrary ID does not teach the model a new fruit.

For a fruit absent from the pretrained classes, we need appropriate training or a different model that supports it.

Q31 What if the user uploads a mango image?

The current application will not reliably recognize it as a mango. Depending on the image, the model may miss it or incorrectly classify it as a supported class. The app should not be presented as supporting mango detection.

Q32 Why not train the model from scratch?

Training from scratch generally requires more labeled data, compute, time and careful optimization. Fine-tuning a suitable pretrained detector is often a more practical starting point for a student project.

PART 4 Dataset and computer vision fundamentals

Q33 What is a dataset?

A dataset is a collection of examples used to train, validate or test a model.

For object detection, examples typically contain images and annotations describing object locations and classes.

Q34 What is an annotation?

An annotation specifies where an object is and which class it belongs to.

For a YOLO-format bounding-box label, a typical row looks like:

```
0 0.50 0.45 0.30 0.25
```

This represents a class ID followed by normalized center coordinates, width and height. The meaning of class ID 0 depends on the dataset's class mapping.

NOTE

The example is illustrative, not an annotation from your current project.

Q35 What is a bounding box?

A bounding box is a rectangle that encloses a detected object. In common formats, it is represented by coordinates such as the left, top, right and bottom edges.

The box allows the application to show where a fruit was detected.

Q36 What is image preprocessing?

Preprocessing prepares input data for a model. Depending on the pipeline, it may include resizing, color conversion, normalization and arranging pixel data into the required format.

In our application, the uploaded image is converted to RGB and passed to the YOLO model. Ultralytics handles the model's internal preprocessing.

Q37 What is data augmentation?

Data augmentation creates variations of training images, such as flips, scale changes, color changes or geometric transformations.

It can help a model generalize to varied conditions. It is relevant when training or fine-tuning a model, but it is not a separate training step performed by our current app.

Q38 What are training, validation and test datasets?

- Training set: used to update model weights.
- Validation set: used to tune training choices and compare model versions.
- Test set: used for final evaluation on data kept separate from training decisions.

A strong evaluation avoids leakage between these sets, especially when several images come from the same video or physical scene.

Q39 What is overfitting?

Overfitting occurs when a model learns patterns specific to its training data and performs poorly on new examples.

A model can achieve strong training results but fail on unseen fruit varieties, backgrounds, lighting conditions or camera angles.

Q40 What is generalization?

Generalization is the model's ability to perform well on previously unseen inputs drawn from the intended use case.

A fruit detector should be tested on diverse images rather than only a few easy examples.

Q41 What is inference?

Inference is the process of using a trained model to make predictions on new data.

When the user uploads a photo and clicks “Run detection,” the application performs inference; it is not training the model at that moment.

Q42 What is a neural network?

A neural network is a computational model made of connected layers that learn numerical representations from data.

YOLO uses a deep neural network to extract visual features and predict object locations and classes.

Q43 What are features in computer vision?

Features are visual patterns useful for identifying objects, such as edges, textures, shapes and color patterns. Deep networks learn hierarchical feature representations from training data.

For instance, the overall shape and texture of a banana may help distinguish it from an apple.

Q44 Why is a large dataset helpful?

A diverse, correctly labeled dataset can expose the model to more examples of fruit appearances, sizes, orientations and backgrounds.

However, quantity alone is not enough. Annotation quality, class balance and how closely the dataset matches real-world use also matter.

PART 5 Confidence threshold, accuracy and evaluation

Q45 What is a confidence threshold?

A confidence threshold is the minimum detection confidence required for a prediction to be retained.

The app uses a slider to control the value passed to the model's conf parameter.

For example, with a threshold of 0.25, detections below 25% confidence are filtered out. A threshold of 0.70 is more restrictive.

Q46 Why is the default threshold 0.25?

It is a practical starting point that permits more candidate detections than a higher threshold. It is not a threshold proven optimal for this particular fruit dataset.

The right threshold should be selected based on validation results and the relative cost of missed detections versus false detections.

Q47 What happens if I increase the threshold?

Typically, fewer low-confidence detections are retained. This may reduce false positives, but it may also cause genuine fruits to be missed.

Q48 What happens if I decrease the threshold?

More detections may be retained, including weak predictions. Recall may improve, but false positives can increase.

Q49 What is a confidence score?

It is the model's confidence value associated with a prediction, according to its scoring mechanism.

A score of 0.92 does not automatically mean the detector is 92% accurate. Confidence scores and measured model accuracy are different concepts.

Q50 What is accuracy?

For a simple classification task, accuracy is the proportion of predictions that are correct.

Object detection requires more detailed evaluation because a prediction must have the correct class and a sufficiently accurate location. Precision, recall and mean Average Precision (mAP) are more informative common metrics.

Q51 What is precision?

Precision answers: of the detections made by the model, how many were correct?

Precision = TP / (TP + FP)

- TP = true positives
- FP = false positives

Example: the model predicts ten fruits, and eight predictions match real fruits. Precision is 8/10 = 80%, assuming the detections meet the evaluation criteria.

Q52 What is recall?

Recall answers: of all the real objects that should have been detected, how many did the model find?

Recall = TP / (TP + FN)

FN means false negatives.

Example: if ten fruits are present and the model detects eight correctly, recall is 80%, assuming the evaluation matching rules are satisfied.

Q53 What is the difference between precision and recall?

Precision measures how trustworthy the predictions are. Recall measures how many actual objects were found.

In fruit inventory, missed fruits may be costly, so recall matters. In an automated system where false detections trigger actions, precision also matters.

Q54 What is IoU?

Intersection over Union measures the overlap between a predicted bounding box and a ground-truth box.

IoU = Intersection area / Union area

If the predicted box closely overlaps the correct box, IoU is high. IoU is commonly used to determine whether a detection matches a labeled object during evaluation.

Q55 What is mAP?

Mean Average Precision summarizes object-detection performance across classes using precision-recall curves, under a specified evaluation protocol.

Common metrics include:

- mAP@0.50
- mAP@0.50:0.95

The latter averages results across multiple IoU thresholds.

> **NOTE**
> For this project: do not quote an mAP value unless you have evaluated the model against a labeled test dataset.

Q56 What is a confusion matrix?

A confusion matrix compares predicted classes with actual classes. It can reveal patterns such as apples being confused with oranges.

For object detection, evaluation must first match predictions to ground-truth objects, so the matrix depends on the chosen matching and confidence criteria.

Q57 How would you measure this project's actual accuracy?

I would prepare a labeled test dataset with ground-truth boxes and class labels, run the model at a documented threshold, and evaluate precision, recall, mAP and per-class performance. I would also measure inference time on specified hardware.

That would give a defensible answer rather than relying on a few visual examples.

PART 6 Counting logic and difficult edge cases

Q58 How does the system count fruits?

The application loops through the detections returned by YOLO and counts the class names.

Simplified example:

```
for box in result.boxes:
    class_id = int(box.cls.item())
    fruit_name = result.names[class_id]
    counts[fruit_name] += 1
```

Every retained bounding box contributes one count to its predicted class.

Q59 Is the count always correct?

No. Counting depends on detection quality.

If a fruit is missed, the count may be too low. If the model produces a false positive or multiple boxes for one object, the count may be too high.

Q60 What happens if fruits overlap?

Occlusion can make objects difficult to recognize. The model may detect both fruits, detect only the visible one, or produce incorrect boxes.

Performance depends on how well similar situations are represented in the training data and on the model's detection capabilities.

Q61 Can the model detect small fruits?

It may detect them if enough visual information is available, but small objects are challenging. Higher input resolution or a suitable model fine-tuned on small fruits may help, though these changes can increase computational cost.

Q62 What if a fruit is partially hidden?

The model may still detect it if enough visual features remain visible. Otherwise, it may miss the fruit or predict an incorrect class.

Q63 Does color alone determine the fruit class?

No. Color can be informative, but shape, texture, context and other learned visual patterns also matter.

A red apple and a green apple should ideally both be recognized as apples, even though their colors differ.

Q64 How does the system prevent duplicate detections?

Modern YOLO pipelines generally apply post-processing, including non-maximum suppression (NMS) in applicable configurations, to reduce overlapping duplicate predictions.

However, duplicates can still occur in difficult scenes. If asked about the exact implementation, describe the behavior of the selected Ultralytics model and configuration rather than claiming the app contains a custom deduplication algorithm.

Q65 What is non-maximum suppression?

NMS compares overlapping candidate boxes and suppresses lower-scoring duplicates according to overlap and scoring rules.

It helps avoid counting the same object multiple times, although it cannot guarantee perfect deduplication in every scene.

Q66 Can the app count unique fruits across a video?

Not reliably in the current implementation.

The video processor detects objects in each frame independently. It does not assign persistent IDs to fruits.

If the same apple appears in 100 frames, it can be detected in many of those frames. Summing frame detections would therefore not give the number of unique apples in the video.

Q67 What does the video count represent?

The current UI reports the maximum simultaneous fruit count per class observed in any processed frame, not the number of unique fruits across the entire video.

Example:

- Frame 1: two apples.
- Frame 2: four apples.
- Frame 3: three apples.

The maximum simultaneous apple count is four.

Q68 What is the difference between object detection and object tracking?

Detection identifies objects within individual frames.

Tracking associates detections across frames, typically assigning persistent IDs so the system can follow an object over time.

For unique fruit counts in video, I would add an appropriate tracker and define rules for when an object is considered new or already counted.

Q69 How would you improve video counting?

I would use a tracking algorithm, such as a supported ByteTrack- or BoT-SORT-based pipeline, associate detections across frames and count each tracked ID once.

For conveyor-belt counting, I might instead count tracked objects crossing a defined line, with safeguards against repeated crossings.

PART 7

Streamlit and application architecture

Q70 Why did you use Streamlit?

Streamlit allows Python developers to create interactive data and ML applications without building a separate frontend framework from scratch.

It provides file uploads, buttons, sliders, layouts, metrics and download controls, which are useful for presenting model predictions.

Q71 Why not use React or HTML/CSS?

Those are also valid options. A custom React frontend could provide more design freedom and a more complex production architecture, but it would require additional frontend development.

Streamlit was a practical choice for building the working ML interface quickly.

Q72 What is the role of OpenCV?

OpenCV is used for image and video processing. In this project, it converts the annotated image into the required color format, reads video frames, writes processed video and encodes annotated images for downloading.

Q73 What is the role of Pillow?

Pillow opens uploaded images and converts them into RGB format for the detection pipeline.

Q74 What is the role of NumPy?

NumPy represents image pixels as arrays. The application converts the PIL image into a NumPy array before passing it to YOLO.

Q75 What is the role of Ultralytics?

Ultralytics provides the YOLO model interface used by the application. It loads the checkpoint, performs inference and returns structured detection results.

Q76 Why use @st.cache_resource?

```
@st.cache_resource
def load_model():
    return YOLO("yolo26n.pt")
```

This caches the loaded model as a reusable resource. It helps avoid repeatedly loading the model during Streamlit reruns, reducing startup overhead.

Q77 Why is model caching important?

Loading model weights and initializing inference components can take time and consume memory. Reusing the loaded model makes interactions more efficient.

Q78 What does st.file_uploader() do?

It provides a user interface for uploading a file. The application restricts supported extensions for images or videos.

The extension filter is a user-interface check, not a complete security validation of file contents.

Q79 What does st.session_state do?

It stores selected values across Streamlit reruns within a user's session.

In the image workflow, the annotated result and counts are stored so that they can remain visible after the detection button triggers a rerun.

Q80 What does st.columns() do?

It lays out content in columns. The image page uses columns to compare the original image with the annotated result and to show multiple metrics side by side.

Q81 How does the download button work?

The application encodes the annotated image into JPEG bytes or reads the processed video file into bytes, then supplies those bytes to Streamlit's download button.

Q82 How is the video processed?

The application opens the uploaded video using OpenCV, reads frames in a loop, runs YOLO on each frame, draws annotations and writes each annotated frame to an MP4 output file.

It also updates the progress bar when a total frame count is available.

Q83 Why is video processing slower than image detection?

A video contains many frames, and the model needs to process each frame. A 30-second video at 30 frames per second contains about 900 frames.

Processing time depends on hardware, resolution, model speed and implementation details.

Q84 What does cv2.VideoCapture() do?

It opens a video source so that the application can read frames.

For an uploaded video, OpenCV uses the temporary file path created by the app.

Q85 What does cv2.VideoWriter() do?

It writes processed frames into an output video. The application configures the frame rate, frame dimensions and MP4 encoding.

Q86 Why does the code use temporary files?

OpenCV video processing works conveniently with file paths. The app writes the uploaded video to a temporary file, processes it, reads the result and removes temporary files after processing.

Q87 Why convert BGR to RGB?

OpenCV commonly uses BGR channel ordering, while PIL and many display workflows use RGB. The conversion ensures the annotated image displays with the correct colors.

Q88 What does result.plot() do?

It generates an annotated image representation of the detection result, usually including bounding boxes and labels.

The application uses this output to display and save detection results.

Q89 Why use a try/finally block for video?

The finally block helps release video resources and remove temporary files even if an error occurs. This reduces the risk of leaving files or resources open.

Q90 What happens if the model file is missing?

The app attempts to load yolo26n.pt. On the first run, Ultralytics may download the model if it is not available locally and internet access permits it.

If loading fails, the application displays an error message and stops the workflow.

PART 8 Code-level questions an expert may ask

Q91 Why do you use classes=FRUIT_CLASSES?

It filters model predictions to the selected class IDs. This prevents other classes in the pretrained model's class mapping from appearing in the application's fruit results.

Q92 Why do you use conf=confidence?

It passes the threshold chosen by the user to the model's prediction function, controlling which detections are retained.

Q93 Why call .item() on the class and confidence tensors?

The model returns tensor-based values. .item() extracts a single scalar value from a one-element tensor, allowing the code to convert class IDs and scores into ordinary Python values.

Q94 Why use Counter?

Counter is a Python utility for counting occurrences of hashable values. Here it conveniently counts how many detections belong to each fruit class.

Q95 Why define make_count_rows()?

It creates a consistent class-wise count table and keeps the display order fixed: apple, banana and orange.

It also ensures a class with zero detections can appear with a count of zero.

Q96 Why use model.predict() instead of training methods?

model.predict() performs inference on new data. Training requires a separate training workflow with a dataset, training configuration and validation.

The current application only uses inference.

Q97 Why is the confidence slider between 0.10 and 0.90?

It provides a user-adjustable range that is practical for experimenting with stricter or more permissive filtering. The exact range is a UI design decision, not a mathematical requirement of YOLO.

Q98 What happens if an image has no detected fruits?

The result should contain zero detections and zero counts for the configured classes. The application should still display the image and make the result understandable.

Q99 What if a corrupted video is uploaded?

The app checks whether OpenCV can open the video. If it cannot, it raises an error and displays a failure message.

In a production version, I would add more file validation, clearer user guidance and limits on file size and processing time.

Q100 What would you change in the code first?

I would improve robustness by validating inputs and handling unsupported or corrupted files, then add evaluation logging and a more comprehensive results report. For expanded fruit support, I would integrate and validate a custom-trained model.

PART 9 Technology stack and environment

Q101 What is Python's role in the project?

Python connects the model, image processing, video processing, counting logic and user interface. It also provides access to the main ML libraries used in the application.

Q102 Why use Python for AI/ML?

Python has a large ecosystem for machine learning, data handling and computer vision. Libraries such as PyTorch, Ultralytics, NumPy and OpenCV reduce the amount of low-level code required.

Q103 What is PyTorch?

PyTorch is a machine-learning framework used to build and execute neural networks. Ultralytics uses PyTorch for supported YOLO model workflows.

Q104 What is a virtual environment?

A virtual environment isolates a project's Python dependencies from those of other projects. This helps prevent version conflicts and makes setup more reproducible.

Q105 What is requirements.txt?

It lists Python packages needed to install the application environment.

For example, this project depends on Streamlit, Ultralytics, OpenCV, Pillow and NumPy.

For stronger reproducibility, package versions should be tested and pinned where appropriate.

Q106 What is the difference between a library and a framework?

A library provides functionality that your code calls when needed. A framework often provides a larger structure or execution model for the application.

The distinction can vary by context. For this project, Streamlit supplies the app interface and execution framework, while NumPy provides array operations.

Q107 Does the project need a database?

The current implementation does not require a database. It processes the uploaded files and displays results during the session.

A future version could use a database to save detection history, reports, user accounts and metadata.

Q108 Does it require a GPU?

Not necessarily. The model can run on a CPU if the installed dependencies and runtime support it. A compatible GPU can accelerate inference, particularly for video, but actual performance depends on the hardware and model configuration.

Q109 How can you deploy the app?

Possible options include a suitable hosted Python application platform or a server configured with the required dependencies and resources.

A deployed version would need file-size limits, resource monitoring, dependency management and security controls.

Q110 Is the app real-time?

The current camera workflow uses st.camera_input, which captures a snapshot and then processes that image. It is not continuous live-camera detection.

Video processing is also an uploaded-video workflow, not a live streaming detector.

PART 10

Non-technical presentation and business questions

Q111 Explain the complete project in one minute.

SAY THIS

“My project, FruitVision AI, is a computer vision application that detects and counts selected fruits in images and videos. It uses a pretrained YOLO26n model for object detection, while Streamlit provides the interactive interface. Users can upload an image, view the annotated output, inspect class-wise counts and download the result. The application also supports uploaded-video processing and camera snapshots. The current version focuses on apples, bananas and oranges, and the next stage would be to train and evaluate a fruit-specific model for more classes and improve video tracking.”

Q112 Explain the workflow without technical terms.

SAY THIS

“The user uploads a photo or video. The AI examines the visual content, identifies fruits it recognizes, marks their locations and counts the detections by fruit type. The application then displays the results so the user can inspect and download them.”

Q113 What is the real-world benefit?

It reduces repetitive manual counting in suitable scenarios and demonstrates how computer vision can assist visual inventory tasks.

The practical benefit would need to be measured by comparing the system with manual counts on representative real-world data.

Q114 Can this replace human workers?

It should not be presented as a complete replacement. It could assist people with repetitive visual counting, while humans handle exceptions, quality checks and situations where the model is uncertain.

Q115 How would you turn it into a commercial product?

I would first validate it on real-world fruit images and measure performance. Then I would improve the model for more fruit classes, add video tracking if needed, save results to a database, deploy it securely, and test it under the actual lighting, camera and workflow conditions.

Q116 What are the risks of using this project in a warehouse?

Potential issues include missed fruits, false counts, occlusion, poor lighting, inconsistent camera angles and domain differences between training images and real warehouse scenes.

A deployment would need validation, monitoring and a process for correcting incorrect results.

Q117 What is the future scope?

Possible improvements include:

- Supporting more fruit categories through fine-tuning.
- Using a labeled test dataset to report measured accuracy.
- Tracking fruit identities across video frames.
- Counting objects crossing a conveyor-belt line.
- Saving reports and detection history.
- Adding batch image processing and structured CSV reports.
- Optimizing performance for the target hardware.
- Building a deployment workflow with logging and error monitoring.

Q118 What is your biggest learning from this project?

SAY THIS

“The main learning was that building an AI application involves more than loading a model. It also requires input handling, inference, post-processing, result presentation, evaluation and clear communication of the model's limitations.”

Q119 What would you do with more time?

SAY THIS

“I would prioritize collecting or selecting a suitable labeled dataset, fine-tuning the pretrained model, evaluating it on unseen images and integrating the trained weights. Then I would improve video tracking and add a downloadable structured report.”

Q120 Why should we consider this project successful if it uses a pretrained model?

SAY THIS

"Using a pretrained model is a legitimate way to build an inference application. The project demonstrates how to integrate a model into an end-to-end workflow. Its current scope is a functional prototype, not a claim that I developed a new detection architecture or trained a state-of-the-art fruit model."

PART 11 Advanced questions from a trained AI/ML expert

These questions are more likely if the examiner wants to go beyond basic project explanations.

Q121 What is the difference between a model's confidence threshold and the IoU threshold?

The confidence threshold controls which predictions are retained based on their scores. The IoU threshold measures box overlap and is used in operations such as NMS or evaluation matching.

They serve different purposes.

Q122 What is the difference between NMS IoU and evaluation IoU?

NMS IoU determines how much overlap between candidate boxes triggers suppression. Evaluation IoU determines whether a predicted box matches a ground-truth box under the evaluation protocol.

The thresholds can be different.

Q123 What is class imbalance?

Class imbalance occurs when some classes have many more examples than others.

A model may perform better on frequent classes than on rare ones. A multi-fruit dataset should be checked for per-class sample counts and performance.

Q124 What is domain shift?

Domain shift occurs when the input data differs from the data on which the model was trained.

For example, internet images may have clean backgrounds, while warehouse camera images may contain poor lighting, motion blur and overlapping fruit. The pretrained model may behave differently in those conditions.

Q125 How would you handle class imbalance during training?

I would inspect per-class data distribution and per-class evaluation metrics, then consider collecting more representative examples, applying suitable augmentation or using training strategies supported by the selected framework.

I would validate any change on held-out data rather than assuming it improves performance.

Q126 What is batch inference?

Batch inference processes multiple images together rather than invoking the model separately for each image. It can improve throughput on suitable hardware, but it requires more memory and careful handling of different input sizes.

The current image workflow processes one uploaded image at a time.

Q127 How would you optimize inference speed?

Potential approaches include selecting a suitable model size, reducing unnecessary image processing, batching inputs where appropriate, using compatible GPU acceleration and evaluating optimized inference formats.

I would measure latency and accuracy before choosing an optimization.

Q128 What is quantization?

Quantization represents model parameters or computations with lower numerical precision, such as integer formats where supported.

It can reduce model size and improve inference efficiency on suitable hardware, but it may affect accuracy and requires compatibility testing.

Q129 What is model drift?

Model drift refers to changes in real-world data or performance over time. For example, camera changes, new fruit varieties or different packaging could reduce detection quality.

A deployed system should be monitored and periodically evaluated on representative data.

Q130 How would you explain false positives and false negatives to a business user?

- False positive: the system reports a fruit where there is no fruit of that class.
- False negative: the system misses a real fruit.

For inventory, false positives can overstate stock, while false negatives can understate it.

Q131 How would you test reliability?

I would create a representative test set covering different fruit sizes, backgrounds, lighting, occlusion and camera conditions. I would measure per-class precision, recall, mAP, latency and count error.

I would also test malformed files, empty scenes and repeated video frames.

Q132 How would you calculate counting error?

For images with ground-truth counts, one basic measure is absolute count error:

Absolute Count Error = | Predicted Count - Actual Count |

Example: actual count = 12, predicted count = 10, so absolute count error = 2.

For a dataset, we could report mean absolute error (MAE) across images. We should separately inspect per-class count error as well.

Q133 What is MAE?

Mean Absolute Error is the average absolute difference between predictions and ground truth.

MAE = (1 / N) x sum of | y_i - y-hat_i |, for i = 1 to N

Here, y_i is the actual count and y-hat_i is the predicted count for example i.

Q134 What is reproducibility in ML?

Reproducibility means another person can recreate the experiment under documented conditions. This includes the dataset version, model weights, software versions, configuration, random seeds where relevant, hardware and evaluation procedure.

A requirements.txt helps, but by itself does not fully guarantee reproducibility.

Q135 How would you prevent data leakage?

I would split the dataset carefully, keeping related images or frames from the same video or scene in the same split where appropriate. This reduces the chance that nearly identical examples appear in both training and test data.

PART 12

Challenging questions that test honesty

These are worth practising because a strong examiner may deliberately challenge the project's claims.

Q136 “You said fruit detection. Why did you not train a fruit-specific model?”

SAY THIS

“My current objective was to implement the YOLO-based detection workflow and application. I used pretrained weights to establish a functional prototype. A fruit-specific model is a planned improvement because it requires preparing labeled data, training and evaluating the model.”

Q137 “How do you know your model is accurate?”

SAY THIS

“I have not established a formal accuracy figure for this version using a labeled test dataset. I can demonstrate the outputs, but to quantify accuracy I would need ground-truth annotations and metrics such as precision, recall, mAP and count error.”

NOTE

This answer shows technical maturity. A demonstration is not the same as a benchmark.

Q138 “Can it count fruits in a video without counting the same fruit repeatedly?”

SAY THIS

“Not uniquely in the current version. It reports the maximum simultaneous count per class observed in a frame. Unique counting across the video would require tracking objects across frames or using a defined counting line.”

Q139 “Why are you using the word AI if you did not train the model?”

SAY THIS

“AI applications can use pretrained models. My work is in integrating the pretrained detector into a complete application workflow. I would distinguish that from developing or training the model architecture myself.”

Q140 “What happens if the model detects an apple as an orange?”

SAY THIS

“That is a classification error. We would need to evaluate per-class performance, examine examples where the classes are confused and potentially fine-tune the model on representative labeled data.”

Q141 “How would you prove your project is better than manual counting?”

SAY THIS

“I would run both methods on the same representative set of images and videos, compare count errors, measure processing time and assess the frequency of human correction. I would not claim it is better until the results support that conclusion.”

Q142 “Why not use a larger YOLO model?”

SAY THIS

“A larger model may offer improved detection quality in some conditions but can require more compute and increase latency. I would compare suitable model sizes on the target data and hardware before making a decision.”

Q143 “Why not use a foundation model or vision-language model?”

A vision-language model may provide flexible semantic understanding, but a dedicated object detector can be more practical for repeated box-level detection and counting tasks. The best choice depends on class coverage, latency, accuracy and deployment constraints.

Q144 “Can this system recognize every fruit?”

SAY THIS

“No. The current implementation is configured for three classes. Supporting more fruits requires a model whose trained classes include them, plus corresponding integration and testing.”

Q145 “What is the biggest technical limitation?”

SAY THIS

“The biggest functional limitation is the narrow class coverage and lack of a custom fruit-specific evaluation. For video, the other important limitation is that detections are not linked to persistent object identities.”